Data Mining

METHODS: DECISION TREES

BY: IVETA MRÁZOVÁ

METHODS: DECISION TREES

Contents:

- Motivation and the TDIDT algorithm
- ID3 - Algorithm
- Other decision tree models
- Ensemble methods

Contents:

- Motivation and the TDIDT algorithm
  - Top Down Induction of Decision Trees
  - Example
  - Important factors
- ID3 - Algorithm
- Other decision tree models
  - Algorithms C4.5 and C5.0
  - CART ~ Classification And Regression Trees
  - CHAID ~ CHI-square Automatic Interaction Detection
- Ensemble methods
  - Bagging ~ bootstrapped aggregating
  - Random forests
  - Boosting and the AdaBoost algorithm

METHODS: DECISION TREES

Contents:

- **Motivation and the TDIDT algorithm**
- *ID3 - Algorithm*
- *Other decision tree models*
- *Ensemble methods*

Contents:

- **Motivation and the TDIDT algorithm**
  - **Top Down Induction of Decision Trees**
  - **Example**
  - **Important factors**
- *ID3 - Algorithm*
- *Other decision tree models*
  - *Algorithms C4.5 and C5.0*
  - *CART ~ Classification And Regression Trees*
  - *CHAID ~ CHI-square Automatic Interaction Detection*
- *Ensemble methods*
  - *Bagging ~ bootstrapped aggregating*
  - *Random forests*
  - *Boosting and the AdaBoost algorithm*

Decision trees – motivation

Indian call centers for the poorer customers only

Affluent account holders at Barclays bank will be connected to British call centers while struggling customers will get Indian ones.

Barclays has found a new route to cut cost and make more profit - outsource jobs to India. Struggling bank customers will be connected to Indian call centers while more affluent account holders will get one in Britain. A new banking phone system will identify those with low credit approvals and put them through to India, the *Daily Mail* reported.

A cap has been fixed. According to the report, those who have considerable savings or a credit limit that allows them to borrow around £500 from the bank or buy its products will be connected to a British operator.

The bank hopes that the new line will filter out their customers who do not have the money to buy the bank's products so their British staff will potentially be able to sell to every caller.

Source: https://www.dnaindia.com/business/report-indian-call-centres-for-the-poorer-customers-only-1358853

Decision trees – motivation

Indian call centers for the poorer customers only

Affluent account holders at Barclays bank will be connected to British call centers while struggling customers will get Indian ones.

… continued from previous slide

The call center staff have been briefed that it will start on April 1, the paper said. "Before we've had people ringing up who have been overdrawn, so we haven't been able to sell them anything," a Barclays sales executive said. The idea, however, has drawn flak.

Critics say the bank is only interested in people with cash and not the genuine concerns of millions who have queries about the state of their finances.

"This is an apartheid system and certainly preferential to their wealthier customers. This is not intended to give the best advice, but a chance to make more money," Eddy Weatherill of the Independent Banking Advisory Service was quoted as saying. A bank spokesman said: "Customers will speak to different teams in different locations depending upon the nature of their query."

Source: https://www.dnaindia.com/business/report-indian-call-centres-for-the-poorer-customers-only-1358853

Decision trees

- Solution of classification tasks
- The built tree models the classification process
  - Efficient building of trees
  - Divides the feature space into rectangular regions
- Classification of patterns according to the region they belong to

Decision trees (2)

Definition:

Let *D* be a database $D = \{\vec{t}_1, \ldots, \vec{t}_p \,;\, 1 \leq i \leq p \,;\, \vec{t}_i = (t_{i1}, \ldots, t_{in})\}$.
Further, let us have the attributes $\{ A_1, A_2, \ldots, A_n \}$ and a set of class labels $C = \{ C_1, \ldots, C_m \}$.

A decision tree for *D* is a tree, where:

- Each inner node is labeled by an attribute A_a ; $1 \leq a \leq n$
- Each edge is labeled by a predicate (applicable to the attribute of the parent node)
- Each leaf has a class label C_j

Decision trees (3)

A solution of classification tasks by means of decision trees requires two steps to be done:

- Induction of the decision tree
  - according to the available training data
- $\forall\ \vec{t}_i \in D$ apply the built decision tree and determine the class label

Advantages:

- Easy to implement and interpret
- Efficient
- Extraction of simple rules
- Applicable also to big data sets

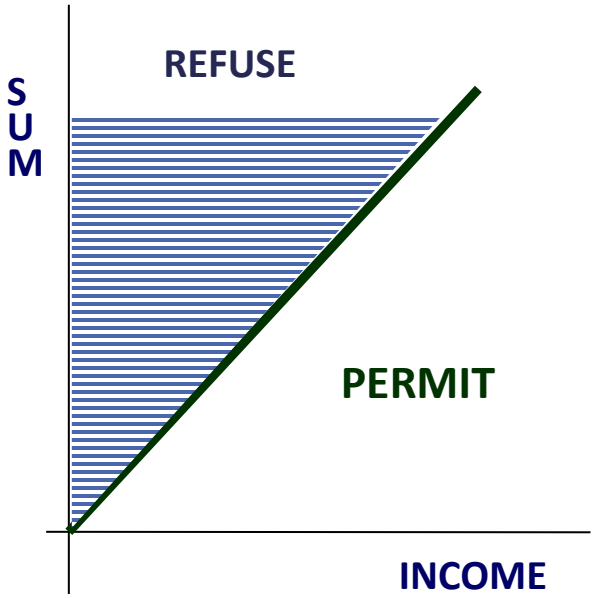
Decision trees (4)

Disadvantages:

- Difficult to process continuous data (attribute categorization ~ divide the feature space into rectangular regions)

  → cannot be always used

  - Example: **Bank loans**
- Difficult to process for missing data
- Over-fitting → PRUNING
- Mutual correlations among the attributes are not considered



Decision trees (5)

Decision attributes for the division of the data set:

- Label the decision nodes of the built tree

Decision predicates:

- Label the edges of the built tree

Stopping criterion:

- E.g., all patterns from the reduced set belong to the same class

Decision trees (6)

Induction of a decision tree – the algorithm:

INPUT: D // training data

OUTPUT: T // decision tree

Building of a decision tree:

// a generic algorithm

T = {};

determine the best criterion for division;

T = form a root node and label it by a decision attribute;

T = add an edge for each decision predicate and label it;

Decision trees (7)

Induction of a decision tree – the algorithm:

// continued

FOR each edge DO

D´ = the data base formed due to the application of the decision predicate to D;

IF the stopping criterion is fulfilled for the given path

THEN

T´ = create a leaf and label it by the corresponding class;

ELSE

T´ = generate a decision tree (D´);

T = add T´ to the edge

Decision trees (8)

The algorithm TDIDT:

(~ Top Down Induction of Decision Trees)

- Induction of the trees by a top-down method (divide-and-conquer)

- **The algorithm TDIDT:**
  1. Choose one attribute as a root of the partial (sub)tree
  2. Divide the data arriving at this node into subsets according to the values of the chosen attribute and add a node for each subset
  3. If there is a node such that the data patterns arriving at it do not belong all to the same class, repeat the process for it starting from step 1;

     else stop

Decision trees (9)

The choice of an attribute applicable to branch the tree:

- The objective: choose such an attribute, that would best divide the patterns from different classes
- Entropy ~ measure of disorder in the system

$$H = -\sum_{j=1}^{m} \left(p_j \log_2 p_j \right)$$

p_j … probability of occurrence of class ***j*** (~ relative frequency of class ***j*** evaluated on the respective set of samples)

m … the number of classes

(Notice: $\log_b x = \log_a x / \log_a b$ with $\log_2 x = \log_{10} x / 0.301$; $0 \log_2 0 = 0$ by df.)

Decision trees (10)

The evaluation of entropy for one attribute:

- For each possible value ***v*** of the considered attribute ***A*** , compute the entropy ***H (A (v))*** on a set of examples, that is covered by the category ***A (v)***

$$H\big(A(v)\big) \;=\; -\sum_{j=1}^{m} \frac{n_j\big(A(v)\big)}{n\big(A(v)\big)} \log_2 \frac{n_j\big(A(v)\big)}{n\big(A(v)\big)}$$

- Compute the **weighted average entropy** ***H (A)*** as a weighted sum of entropies ***H (A (v))***

$$H(A) \;=\; \sum_{v \in Val(A)} \frac{n\big(A(v)\big)}{n} H\big(A(v)\big)$$

Decision trees (11)

Application of decision trees to classify new patterns:

- The non-leaf nodes of the tree contain the information about the decision attributes used for branching
- The edges of the tree correspond to the values of these attributes
- The leaves of the tree contain the information about the assigned class
- Starting from the root of the tree, there is successively gathered the information about the values of the respective attributes

Decision trees (12)

A transformation of a tree to a set of rules:

- Each path between the root and a leaf of a tree corresponds to a rule
- The non-leaf nodes (feature attributes) will appear (together with their value for the respective edge) on the left-hand side (in the body) of the rule
- The leaf node (target class attribute) will represent the right-hand side (head) of the rule

Decision trees

Example: an application for a loan

Client	Income	Account	Gender	Unemployed	Loan approved
C1	High	High	Female	No	Yes
C2	High	High	Male	No	Yes
C3	Low	Low	Male	No	No
C4	Low	High	Female	Yes	Yes
C5	Low	High	Male	Yes	Yes
C6	Low	Low	Female	Yes	No
C7	High	Low	Male	No	Yes
C8	High	Low	Female	Yes	Yes
C9	Low	Middle	Male	Yes	No
C10	High	Middle	Female	No	Yes
C11	Low	Middle	Female	Yes	No
C12	Low	Middle	Male	No	Yes

Decision trees
Example: an application for a loan (2)

A four-field contingency table for the attributes income and loan approved:

	Loan approved - Yes	Loan approved - No
Income high	5	0
Income low	3	4

The choice of the decision attribute according to minimum entropy:

- **INCOME:**

$$H(INCOME) = \frac{5}{12} H\big(INCOME(HIGH)\big) + \frac{7}{12} H\big(INCOME(LOW)\big) =$$

$$= \frac{5}{12} \cdot 0 + \frac{7}{12} \cdot 0.9852 = 0.5747$$

Decision trees
Example: an application for a loan (3)

The choice of the decision attribute according to minimum entropy:

- **INCOME** (continued):

$$H\big(INCOME(HIGH)\big) = -p_+ \log_2 p_+ - p_- \log_2 p_- =$$

$$= -\frac{5}{5}\log_2\frac{5}{5} - \frac{0}{5}\log_2\frac{0}{5} = 0 + 0 = 0$$

$$H\big(INCOME(LOW)\big) = -p_+ \log_2 p_+ - p_- \log_2 p_- =$$

$$= -\frac{3}{7}\log_2\frac{3}{7} - \frac{4}{7}\log_2\frac{4}{7} = 0.9852$$

Decision trees
Example: an application for a loan (4)

The choice of the decision attribute according to minimum entropy:

- **ACCOUNT:** $H(ACCOUNT) = \frac{4}{12} H\big(ACCOUNT(HIGH)\big) +$

$$+ \frac{4}{12} H\big(ACCOUNT(MIDDLE)\big) +$$

$$+ \frac{4}{12} H\big(ACCOUNT(LOW)\big) =$$

$$= \frac{1}{3} \cdot 0 + \frac{1}{3} \cdot 1 + \frac{1}{3} \cdot 1 = 0.6667$$

Decision trees
Example: an application for a loan (5)

The choice of the decision attribute according to minimum entropy:

- **GENDER:**

$$H(GENDER) = \frac{6}{12} H\big(GENDER(MALE)\big) +$$

$$+ \frac{6}{12} H\big(GENDER(FEMALE)\big) =$$

$$= \frac{1}{2} \cdot 0.9183 + \frac{1}{2} \cdot 0.9183 = 0.9183$$

Decision trees
Example: an application for a loan (6)

The choice of the decision attribute according to minimum entropy:

- **UNEMPLOYED:**

$$H(UNEMPLOYED) = \frac{6}{12} H\big(UNEMPLOYED(YES)\big) +$$

$$+ \frac{6}{12} H\big(UNEMPLOYED(NO)\big) =$$

$$= \frac{1}{2} \cdot 1 + \frac{1}{2} \cdot 0.65 = 0.825$$

Decision trees
Example: an application for a loan (7)

The choice of the decision attribute according to minimum entropy:

→ The first branching for the attribute **INCOME**: 2 classes

- INCOME (HIGH): LOAN APPROVED (YES) for all patterns
- INCOME (LOW): 7 clients + branching
  - **ACCOUNT:**

$$H(ACCOUNT) = \frac{2}{7}H\big(ACCOUNT(HIGH)\big) + \frac{3}{7}H\big(ACCOUNT(MIDDLE)\big) + \\ + \frac{2}{7}H\big(ACCOUNT(LOW)\big) = 0.3935$$

  - **GENDER:** ***H (GENDER) = 0.965***
  - **UNEMPLOYED:** ***H (UNEMPLOYED) = 0.9792***

Decision trees
Example: an application for a loan (8)

The choice of the decision attribute according to minimum entropy:

→ The second branching according to the attribute **ACCOUNT**: 3 classes

- ACCOUNT (HIGH): LOAN APPROVED (YES) for all patterns
- ACCOUNT (LOW): LOAN APPROVED (NO) for all patterns
- ACCOUNT (MIDDLE): 3 clients + branching
  - **GENDER:**

$$H(GENDER) = \frac{2}{3}H\big(GENDER(MALE)\big) + \frac{1}{3}H\big(GENDER(FEMALE)\big) =$$

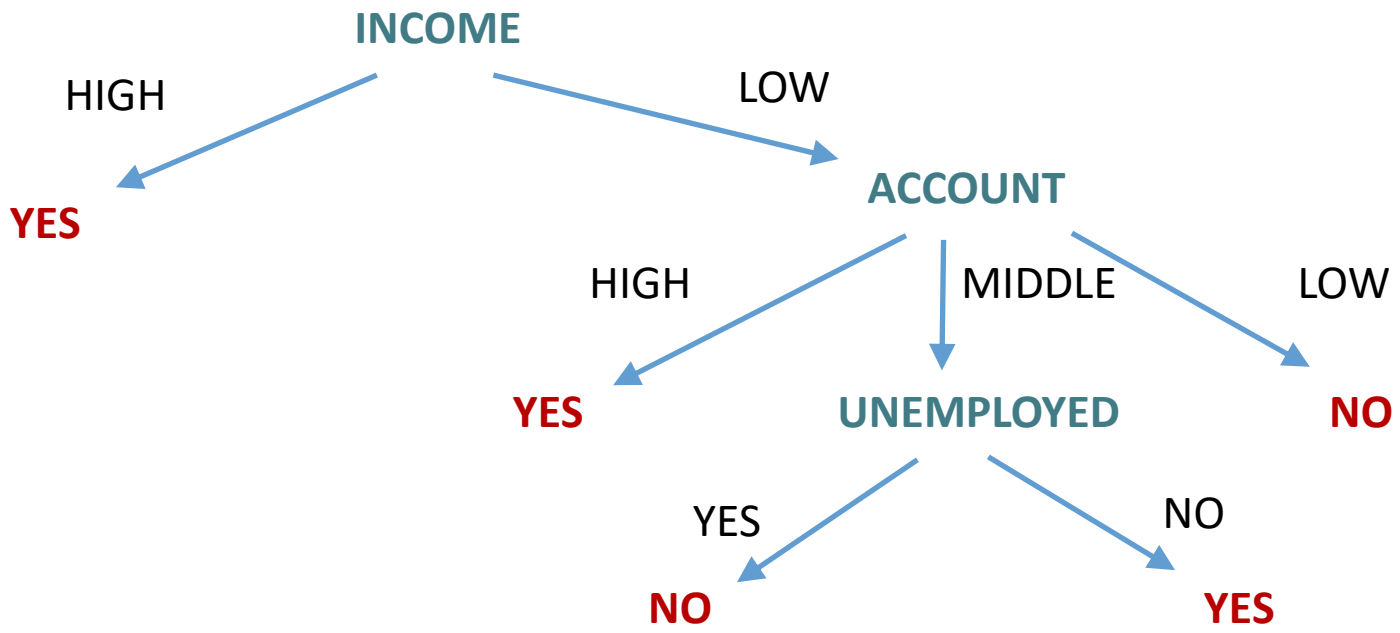
$$= \frac{2}{3}\cdot 1 + \frac{1}{3}\cdot 0 = 0.6667$$

  - **UNEMPLOYED:** ***H (UNEMPLOYED) = 0***

→ The third branching according to the attribute **UNEMPLOYED**

Decision trees
Example: an application for a loan (9)

The induced decision tree:



Decision trees
Example: an application for a loan (10)

A transformation of the induced tree to rules:

IF *INCOME (HIGH)* **THEN** *LOAN APPROVED (YES)*

IF *INCOME (LOW)* $\wedge$ *ACCOUNT (HIGH)*
THEN *LOAN APPROVED (YES)*

IF *INCOME (LOW)* $\wedge$ *ACCOUNT (LOW)*
THEN *LOAN APPROVED (NO)*

IF *INCOME (LOW)* $\wedge$ *ACCOUNT (MIDDLE)* $\wedge$
UNEMPLOYED (YES) **THEN** *LOAN APPROVED (NO)*

IF *INCOME PRIJEM (LOW)* $\wedge$ *ACCOUNT (MIDDLE)* $\wedge$
UNEMPLOYED (NO) **THEN** *LOAN APPROVED (YES)*

Decision trees (13)

Factors important for the induction of a decision tree:

- **The choice of the decision attributes**
  - The impact on the efficiency of the built tree
  - ´informed´ expert entry for the considered area
- **The order of the decision attributes**
  - Eliminate redundant comparisons
- **Branching**
  - The number of necessary divisions
  - More demanding for continuous values or many values

Decision trees (14)

Factors important for the induction of a decision tree:

- **The form of the built tree**
  - More suitable are balanced trees with as few levels as possible x more complex comparisons
  - Some algorithms create binary trees only (often quite deep)

- **Stopping criteria**
  - Correct classification of the training data
  - Early stopping prevents building of large trees and overtraining
    → **accuracy x efficiency**
  - **Occam´s razor** (William of Occam – 1320)
    preference of the simplest hypothesis for D

Decision trees (15)

Factors important for the induction of a decision tree:

- **Training data and their quantity**
  - Too few → the formed tree might be not reliable enough for more general data
  - Too many → the danger of overtraining
- **Pruning**
  - A higher accuracy and efficiency for the tested data
  - Removal of redundant comparisons, or entire subtrees, and irrelevant attributes, resp.
  - Overtraining → removal of entire subtrees at lower levels

Decision trees (16)

Factors important for the induction of a decision tree:

- **Pruning** (continued)
  - Can be done already during tree induction

    → avoids forming of excessively large trees
  - A different approach – pruning of already formed trees
- **Time and space complexity**
  - Depends on the number of training data ***p*** , the number of attributes ***n*** and the form of the induced tree – depth, branching
  - The time complexity of the tree induction: ***O (n p log p)***
  - The time complexity of the classification of ***q*** patterns with a tree of the depth ***O (log p)*** : ***O (q log p)***

METHODS: DECISION TREES

Contents:

- *Motivation and the TDIDT algorithm*
- **ID3 - Algorithm**
- *Other decision tree models*
- *Ensemble methods*

Contents:

- *Motivation and the TDIDT algorithm*
  - *Top Down Induction of Decision Trees*
  - *Example*
  - *Important factors*
- **ID3 - Algorithm**
- *Other decision tree models*
  - *Algorithms C4.5 and C5.0*
  - *CART ~ Classification And Regression Trees*
  - *CHAID ~ CHI-square Automatic Interaction Detection*
- *Ensemble methods*
  - *Bagging ~ bootstrapped aggregating*
  - *Random forests*
  - *Boosting and the AdaBoost algorithm*

Decision trees (17)

ID3 – Algorithm (~ Iterative Dichotomiser, Ross Quinlan, 1986):

- Training of functions with Boolean values
- Greedy method
- Builds the tree in a top-down manner
- In each node, it chooses such an attribute that best classifies local training data

  → the best attribute has the biggest information gain
- The process continues until all the training data are classified correctly, or until all the attributes have been used

Decision trees (18)

ID3 – Algorithm (continued):

- ***EXAMPLES*** ~ training data (patterns)
- ***TARGET_ATTRIBUTE*** ~ attribute, the value of which shall the tree predict (class)
- ***ATTRIBUTES*** ~ the list of attributes to be tested by the built tree
- Returns the decision tree that correctly classifies the provided training data (***EXAMPLES***)
- **Entropy** ~ measure of disorder (~ „surprise“) in the training set
- **Information gain** ~ expected entropy reduction after dividing the data according to the considered attribute
- **Preference of ´smaller´ trees x over-training**

Decision trees (19)

ID3 – Algorithm (continued):

ID3(EXAMPLES, TARGET_ATTRIBUTE, ATTRIBUTES)

- Build a root ***ROOT*** of the tree
- ***IF*** all ***EXAMPLES*** are positive, ***RETURN*** tree ***ROOT*** , that has a single node and is labelled ´***+***´
- ***IF*** all ***EXAMPLES*** are negative, ***RETURN*** tree ***ROOT***, that has a single node and is labelled ´***-***´
- ***IF ATTRIBUTES = {}*** , ***RETURN*** tree ***ROOT*** , that has a single node and is labelled by the most frequent value ***TARGET_ATTRIBUTE*** in ***EXAMPLES***

Decision trees (20)

// ***ID3(EXAMPLES, TARGET_ATTRIBUTE, ATTRIBUTES)***

// continued

ELSE BEGIN

- ***A <=*** attribute from ***ATTRIBUTES*** , that best classifies ***EXAMPLES***
- Decision attribute for ***ROOT <= A***
- For all possible values v_i of the attribute ***A***
  - Attach a new edge to ***ROOT***, that corresponds to the predicate $A = v_i$
  - Let $EXAMPLESv_i$ be a subset of ***EXAMPLES*** with the value v_i for ***A***

Decision trees (21)

// ***ID3(EXAMPLES, TARGET_ATTRIBUTE, ATTRIBUTES)***

// continued

- ***IF EXAMPLESv_i = {}***
  - Attach a leaf to the edge and label it by the most frequent value ***TARGET_ATTRIBUTE*** in ***EXAMPLES***
  - ***ELSE*** attach a subtree ***ID3(EXAMPLESv_i , TARGET_ATTRIBUTE, ATTRIBUTES - {A})*** to the edge
- ***END***
- ***RETURN ROOT***

Decision trees (22)

ID3 – Algorithm (continued):

- Criterion for data division: **information gain**
  - evaluation by means of **entropy**:
    - ***m*** … classes
    - p_j … probability of class ***j***

      (~ number of patterns from class ***j*** / the number of all patterns from the training set ***EXAMPLES***)

$$ENTROPY(EXAMPLES) = \sum_{j=1}^{m} -p_j \log_2 p_j$$

Decision trees (23)

ID3 – Algorithm (continued):

- Criterion for data division: **information gain**
  - **Information gain - GAIN**:
    - ***A*** … the considered attribute
    - ***VALUES(A)*** … set of all possible values for the attribute ***A***

$$GAIN(EXAMPLES, A) = ENTROPY(EXAMPLES) -$$

$$- \sum_{v \in VALUES(A)} \frac{|EXAMPLES_v|}{|EXAMPLES|} ENTROPY(EXAMPLES_v)$$

METHODS: DECISION TREES

Contents:

- *Motivation and the TDIDT algorithm*
- *ID3 - Algorithm*
- **Other decision tree models**
- *Ensemble methods*

Contents:

- *Motivation and the TDIDT algorithm*
  - *Top Down Induction of Decision Trees*
  - *Example*
  - *Important factors*
- *ID3 - Algorithm*
- **Other decision tree models**
  - **Algorithms C4.5 and C5.0**
  - **CART ~ Classification And Regression Trees**
  - **CHAID ~ CHI-square Automatic Interaction Detection**
- *Ensemble methods*
  - *Bagging ~ bootstrapped aggregating*
  - *Random forests*
  - *Boosting and the AdaBoost algorithm*

Decision trees (24)

Algorithm C4.5 (Ross Quinlan - 1993):

- A modification of the ID3-algorithm
- Missing data
  - Ignored during tree construction

    → when evaluating the criterion for data division, only the known values of the given attribute will be considered
  - During classification, the missing values will be estimated based on the values known for other patterns
- Continuous data
  - The data is categorized according to the values of the respective attribute on the patterns from the training set

Decision trees (25)

Algorithm C4.5 (continued):

- Pruning
  - **Validation**: divide the training data into a training set (2/3) and a validation set (1/3)
  - **Replacement of subtrees** (reduced – error pruning) by leaves labeled by the most frequent class in the respective training patterns
    - **The new tree must not perform worse on the validation set than the original one!**
    - Otherwise, the subtree will be not replaced

Decision trees (26)

Algorithm C4.5 (continued):

- Pruning
  - **Subtree raising**
    - A replacement of a subtree by its most frequently used subtree – this subtree will then be raised to a higher level
    - Test the classification accuracy
- Rule post-pruning
  - Inference of a decision tree based on the training set – over-training is allowed
  - Conversion of the built tree to an equivalent set of rules
    - For each path from the root to a leaf, there will be extracted a rule

Decision trees (27)

Algorithm C4.5 (continued):

- Rule post-pruning:
  - Prune (~ generalize) the rules by eliminating all possible assumptions if it does not impact a worse estimated classification accuracy
  - Order pruned rules according to their expected accuracy
    - Then, use the rules in this order to classify the patterns

→ easier and more radical pruning than in the case of the complete decision trees

→ transparent and easy to understand rules

Decision trees (28)

Algorithm C4.5 (continued):

- An estimation of the accuracy of rules and trees, resp.
  - A standard approach - use a validation set
  - **C4.5:**
    - **An evaluation of the accuracy on the training data**
      ~ The ratio of patterns correctly classified by the rule and all patterns covered by the rule $\left(\hat{p} = \frac{p_1}{p}\right)$
    - **An assessment of the standard deviation of the accuracy**
      ~ Binomial distribution is assumed when looking for the probability of obtaining a given number of correct decisions for the considered number of patterns

Decision trees (29)

Algorithm C4.5 (continued):

- An estimation of the accuracy of rules and trees, resp.

  → **Take the lower accuracy bound determined for the chosen confidence interval as the respective rule characteristics**

  - Example: For the confidence interval computed at the 95% confidence level, the lower accuracy bound for new data will correspond to:

ACCURACY_ON_THE_TRAINING_DATA –

– 1.96 x STANDARD_DEVIATION $\left(\sim \hat{p} - 1.96 \cdot \sqrt{\frac{\hat{p}\,(1-\hat{p})}{p}} \right)$

Decision trees (30)

Algorithm C4.5 (continued):

- The criterion for data division: **information gain ratio**

$$GAIN\_RATIO(EXAMPLES, A) =$$

$$= \frac{GAIN(EXAMPLES, A)}{-\sum_{v \in VALUES(A)} \left(\frac{|EXAMPLES_v|}{|EXAMPLES|} \log_2 \frac{|EXAMPLES_v|}{|EXAMPLES|} \right)}$$

- For data division, the attribute with the biggest information gain ratio will be used
  - disadvantages excessive branching

Decision trees (31)

Algorithm C5.0:

- A commercial version of C4.5 for large databases
- A similar induction of the decision tree
- Improved (more efficient) generation of rules
- A higher accuracy is achieved:
  - **Boosting** (~ a combination of several classifiers)
    - Based on the original training data, various training (sub)sets will be built
    - To each training pattern, there is assigned a weight corresponding to its impact during classification
    - For each weight combination, a separate classifiers is formed
    - Each classifier has its own vote during the subsequent classification, the majority (class) wins

Gini-index G: for an attribute with r values $v_1,\ldots, v_r$, n data patterns, and m classes: $G=\sum_{i=1}^{r} n_i G(v_i)/n$ with $G(v_i) = 1- \sum_{j=1}^{m} p_j^2$ and $\sum_{i=1}^{r} n_i = n$ (n_i is the nr. of data patterns taking on the value v_i ; p_j denotes the probability of occurrence of class j in the n_i data patterns).

Decision trees (32)

Classification and regression trees – algorithm CART:

~ Classification And Regression Trees

- Generates binary trees
- The choice of the best decision attribute for data division – entropy, Gini-index (lower values of G imply better discrimination)
- During each data division, just two edges are formed
- Data division is performed according to the ´best´ data point found for the respective node ***t***
- Check all possible values ***v*** of decision attribute ***A***, $v \in VALUES(A)$
  - ***L, R*** …….. left-hand-side, resp. right-hand-side subtree
  - P_L , P_R … probability of processing by the respective subtree

$$P_L = \frac{NUMBER\_OF\_PATTERNS\_IN\_THE\_LEFT\_SUBTREE\_L}{NUMBER\_OF\_PATTERNS\_IN\_THE\_ENTIRE\_TRAINING\_SET}$$

Decision trees (33)

Classification and regression trees (continued):

- Check all possible values ***v*** of the decision attribute ***A***, $v \in VALUES(A)$
  - in the case of equality, the right-hand-side subtree will be used
  - $P(C_j \mid t_L)$, $P(C_j \mid t_R)$ … probability, that the pattern belongs to class C_j and is in the left-hand-side, resp. the right-hand-side subtree (of node **t**)

$$P(C_j|t) = \frac{NUMBER\_OF\_PATTERNS\_FROM\_CLASS\_j\_IN\_THE\_SUBTREE}{NUMBER\_OF\_PATTERNS\_IN\_THE\_CONSIDERED\_NODE}$$

  - The choice of the criterion for data division:

$$\Phi(v|t) = 2P_L P_R \sum_{j=1}^{m} \left| P(C_j|t_L) - P(C_j|t_R) \right|$$

(maximum values for all patterns from the considered class grouped in the same subtree)

Decision trees (34)

Classification and regression trees (continued):

- Characteristic properties of CART:
  - The order of the attributes corresponds to their impact during classification
  - Missing data is ignored – and will be not counted towards the evaluation of the decision criteria
  - The stopping criterion:
    - No further division would improve classification accuracy
  - A high accuracy achieved on the training set does not have to reflect the accuracy on the test data

Decision trees (35)

Algorithm CHAID:

~ Chi-square Automatic Interaction Detection

- The criterion for branching: χ^2
- The algorithm automatically groups together the values of the categorical attributes
  - Attribute values are gradually grouped together starting from the original number till there are just two groups of them only
  - Subsequently, there will be selected such an attribute and its categorization that are the best ones for data division at the given step

→ during branching, there will be not created as many branches as there are attribute values

Decision trees (36)

Algorithm CHAID – grouping of attribute values:

1. Repeat, till there are just two groups of attribute values only
   a) Choose such a pair of attribute categories that are most similar one to the other from the point of view of χ^2 and that can be merged
   b) Consider the new attribute category to be a possible grouping to be performed at the given step
2. For each of the possible groups of values, test their relevance to the predictor variable by means of the χ^2 –test
3. Choose the grouping that yields the ´best´ grouping of attribute values with the most significant split
4. Determine, if this grouping contributes in a statistically significant way to distinguishing the patterns from different classes

METHODS: DECISION TREES

Contents:

- *Motivation and the TDIDT algorithm*
- *ID3 - Algorithm*
- *Other decision tree models*
- **Ensemble methods**

Contents:

- *Motivation and the TDIDT algorithm*
  - *Top Down Induction of Decision Trees*
  - *Example*
  - *Important factors*
- *ID3 - Algorithm*
- *Other decision tree models*
  - *Algorithms C4.5 and C5.0*
  - *CART ~ Classification And Regression Trees*
  - *CHAID ~ CHI-square Automatic Interaction Detection*
- **Ensemble methods**
  - **Bagging ~ bootstrapped aggregating**
  - **Random forests**
  - **Boosting and the AdaBoost algorithm**

Bagging ~ bootstrapped aggregating

Attempts to reduce the variance of the classifier.

- if the variance of a classifier is σ^2, then the variance of the average of k independent and identically distributed (i.i.d.) classifiers reduces to σ^2/k
- bagging works best when the individual classifiers from various ensemble components satisfy the i.i.d. property.
- decision trees are an ideal choice for bagging as they have a low bias and high variance (if they are grown sufficiently deep)

x bagging does not reduce the model bias and for cases in which the bias is the primary source of error, it may even slightly worsen accuracy due to correlations between the ensemble components (and the failed i.i.d. assumption)

Bagging ~ bootstrapped aggregating

Approximation of i.i.d. classifiers in bagging?

~ the data points are sampled uniformly from the original data with replacement

= > bootstrapping:

- a sample is drawn of approximately the same size as the original data
- the drawn sample may comprise duplicates and contains a fraction of roughly $1 - (1 - 1/n)^n \approx 1 - 1/e$ distinct data points from the original data, i.e., 63,2%; e represents the base of the natural logarithm.
- the probability of a data point being not included in a sample is $(1 - 1/n)^n$
- a total of k different bootstrapped samples, each of size n, are drawn independently, and a classifier is trained on each of them
- for a given test instance, the predicted class label is reported by the ensemble as the dominant vote of the different classifiers

Random Forests

- If the k different classifiers, each with variance σ^2, have a positive pairwise correlation of ρ between them, then the variance of the averaged prediction can be shown to be $\rho \cdot \sigma^2 + ((1 - \rho) \cdot \sigma^2) / k$
- The term $\rho \cdot \sigma^2$ is invariant to the number of components k in the ensemble. This term limits the performance gains from bagging.
- Random forests are defined as an ensemble of decision trees, in which randomness has explicitly been inserted into the model building process of each decision tree.
- Random forests can be viewed as a generalization of the basic bagging method, as applied to decision trees. In addition, bootstrapped decision trees tend to be positively correlated.

Random Forests (2)

- Unfortunately, correlated trees limit the amount of error reduction obtained from bagging.

$\Rightarrow$ The idea is to use a randomized decision tree model with less correlation between the different ensemble components.

- The underlying variability can then be more effectively reduced by an averaging approach.

- The obtained results are often more accurate than a direct application of bagging on decision trees

Random Forests (3)

The main idea:

- if one tree is good, then many trees (a forest) should be better, if there is enough variety between the trees
- Introduce randomness to the standard dataset – two options:
  - **Bagging** - make the trees different by training them on slightly different data (bootstrap samples from the dataset for each tree)
  - **Limited number of features** - at each node, a random subset of the features is given to the tree, and it can only pick from that subset rather than from the whole set.

    This also speeds up training due to fewer features to search over at each stage and there is no need to prune the trees

  + Reduced variance does not negatively affect the bias
- Once the set of trees are trained, the output of the forest can be determined as the majority vote for classification.

Random Forests (4)

The Basic Random Forest Training Algorithm:

- For each of ***N*** trees:
  - create a new bootstrap sample of the training set
  - use this bootstrap sample to train a decision tree
  - at each node of the decision tree, randomly select ***m*** features, and compute the information gain only on that set of features, selecting the optimal one
  - repeat until the tree is complete

Parameters:

- the number of features
  - a subset size that is the square root of the number of features seems to be common (random forests are not too sensitive to this parameter)
- the number of trees for the forest
  - building trees can be continued until the error stops decreasing

Boosting

The main principle:

- a weight is associated with each training pattern, and different classifiers are trained using these weights
- the weights of the patterns are iteratively modified based on the performance of the classifier
  - future models to be constructed depend on previous models
  - each classifier is constructed using the same algorithm on a different weighted training set

The idea:

- In future iterations, focus on misclassified patterns by increasing their relative weight
- By iteratively using this approach and creating a weighted combination of the various classifiers, it is possible to create a classifier with a lower overall bias

The AdaBoost algorithm

(Freund & Schapire, 1996)

Algorithm *AdaBoost*(Data Set: ***D***, Base Classifier: ***A***, Maximum Rounds: ***T***)

begin

t = 0;

for each instance ***i*** initialize the weight $W_1(i)=1/n$;

repeat

$t = t + 1$;

Determine weighted error rate ϵ_t on ***D*** when the base algorithm ***A*** is applied to the weighted data set with the weights $W_t(\cdot)$;

$\alpha_t = (1/2) \log_e ((1 - \epsilon_t)/ \epsilon_t)$; $\left(e^{\alpha_t} = \sqrt{1-\epsilon_t/\epsilon_t}\right)$

for each misclassified $X_i \in D$ **do** $W_{t+1}(i) = W_t(i)e^{\alpha_t} = W_t(i) \cdot \sqrt{1-\epsilon_t/\epsilon_t}$;

else (correctly classified instance) **do** $W_{t+1}(i) = W_t(i)\, e^{-\alpha_t} = W_t(i) \cdot \sqrt{\epsilon_t/1-\epsilon_t}$;

for each instance X_i **do** normalize $W_{t+1}(i) = W_{t+1}(i) / [\sum_j W_{t+1}(j)]$, (1≤j≤n);

until $((t \geq T)$ OR $(\epsilon_t = 0)$ OR $(\epsilon_t \geq 0.5))$;

Use ensemble components with weights α_t for test instance classification;

end

ϵ_t is the fraction of incorrectly predicted training instances (computed after weighting with $W_t(i)$) by the model in the *t*-th iteration

The AdaBoost algorithm: Summary

Training

- Produces a sequence of classifiers (with the same base learner)
- Each classifier depends on the previous one, and focuses on the previous one's errors
- Examples that are incorrectly predicted in previous classifiers are given higher weights

Testing

- For a test case, the results of the series of classifiers are combined to determine the final class of the test case
- The actual performance depends on the data and the base learner
- Susceptible to noise => when the number of outliers is very large, the emphasis placed on the hard examples can hurt the performance

Random Forests vs. Boosting

Random Forests:

- random forests are fast - they can run in parallel and search over a small set of features at each stage

$\Rightarrow$ as the trees are cheaper to train, we can grow more of them in the same computational time, even on large and complicated datasets

$\Rightarrow$ can produce surprisingly good results in comparison to the small parts of the problem space seen by each tree

Boosting:

- boosting is exhaustive – searches over the entire feature set at each stage, and each stage depends on the previous one

$\Rightarrow$ boosting must run sequentially, and can be expensive

- for the same number of trees, boosting outperforms random forests (as random forests search only a small subset of the data at each stage)